Lists in Python 3

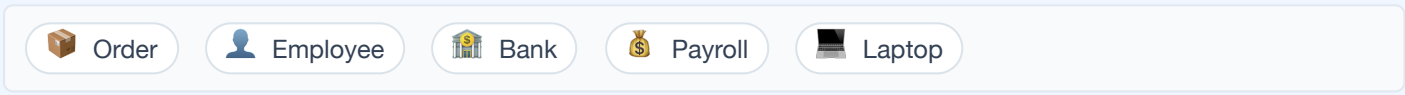
Module: Core Python · Collections
Previous module: Strings
Next module: Tuples and Sets

Module Outcome:

- Keep a Swiggy cart or attendance register in order — add, swap, remove
- Index from `0`, loop items, and know when a list is the right bag
- Mutate in place without accidentally aliasing two carts together

Think of a **list** like a numbered shopping cart or a attendance register:

- Items stay in the **order** you add them (first product added appears first).
- You can **change** items later — swap a wrong name, add a new expense, remove a cancelled order.
- Every item has a **position number** starting from `0` (index).



Where used?

- **Swiggy** — cart items, order line rows
- College apps — marks per subject, attendance registers
- **Employee Management Portal** — employee name rosters, salary line items
- **HDFC Bank** — recent transactions (ordered history)
- **Apollo Hospitals** — appointment queues, task lists

Real-life parallels:

Situation	List in code
E-commerce cart: `[ "Notebook", "Pen", "USB Cable" ]`	Ordered collection of product names
Employee IDs in a team roster	`[101, 102, 105, 108]`
Monthly expenses in rupees	`[450, 1200, 89, 2500]`
Student marks in 5 subjects	`[78, 82, 91, 65, 88]`

Unlike a string, you can **replace** one element without creating a whole new list object (though some operations do create new lists — we will clarify in class).

Concepts and Definitions

Term	Definition
List	Ordered, mutable sequence of items; written with square brackets `[]`.
Element / item	One value inside a list (any type: int, str, float, bool, even another list).
Index	Zero-based position; `0` is first, `-1` is last.
Slice	Sub-list using `[start:stop:step]`; stop is exclusive.
Mutable	Can change contents after creation (`append`, assignment `lst[0] = x`).
Method	Function tied to the list object, e.g. `cart.append("Pen")`.
Nested list	List inside a list — like rows in a spreadsheet.
List comprehension	Compact syntax to build a list: `[expr for item in iterable if condition]`.
CRUD	Create, Read, Update, Delete — basic data operations on collections.

Step-by-Step Notes with Annotated Python 3 Code

Step 1 — Create and read a list

```
# CREATE — empty list or literal
cart = []                                    # empty cart
products = ["Notebook", "Pen", "USB Cable"]  # e-commerce product names
expenses = [450, 1200, 89, 2500]             # rupees spent this month

# READ — by index and slice
print(products[0])       # Notebook  (first item, index 0)
print(products[-1])      # USB Cable (last item)
print(expenses[1:3])     # [1200, 89] — index 1 up to (not including) 3
print(len(products))     # 3 — number of items
```

Note:
- Wrong: Indexing past the end (`lst[10]` on a short list) → `IndexError`.
- Prefer: Check `len(lst)` first.

Print first item, last item, and length of a Swiggy cart list.

```
cart = ["Notebook", "Pen", "USB"]
print(cart[0], cart[-1], len(cart))
```

Expected Output:

```
Notebook USB 3
```

Slice the middle two expenses from a four-item list.

```
expenses = [450, 1200, 89, 2500]
print(expenses[1:3])  # [1200, 89]
```

Expected Output:

```
[1200, 89]
```

Step 2 — Update list items (CRUD: Update)

```
employees = ["Ananya", "Ravi", "Priya"]

# UPDATE single element
employees[1] = "Ravi Kumar"   # rename / correct spelling
print(employees)                # ['Ananya', 'Ravi Kumar', 'Priya']

# ADD items
cart = ["Shirt"]
cart.append("Jeans")            # add one item at end
cart.extend(["Belt", "Socks"])  # add multiple items
cart.insert(1, "T-Shirt")       # insert at index 1
print(cart)                     # ['Shirt', 'T-Shirt', 'Jeans', 'Belt', 'Socks']
```

Note:
- Wrong: Using `append` with a list when you mean `extend` (`lst.append([4, 5])` → one nested element).
- Prefer: `extend([4, 5])` to add multiple items.

Update the second name in a list to Ravi Kumar and print the list.

```
names = ["Ananya", "Ravi", "Priya"]
names[1] = "Ravi Kumar"
print(names)
```

Build a cart with append, extend, and insert at index 0.

```
cart = []
cart.append("Notebook")
cart.extend(["Pen", "USB"])
cart.insert(0, "PROMO-Mug")
print(cart)
```

Expected Output:

```
['PROMO-Mug', 'Notebook', 'Pen', 'USB']
```

Common Error: `cart.append(["Pen", "USB"])` nests a list inside the cart. Use `extend([...])` to add multiple items flat.

Step 3 — Delete items (CRUD: Delete)

```
orders = ["ORD-101", "ORD-102", "ORD-103", "ORD-102"]

# DELETE by value (first match only)
orders.remove("ORD-102")        # removes first occurrence
print(orders)                   # ['ORD-101', 'ORD-103', 'ORD-102']

# DELETE by index — pop returns the removed item
cancelled = orders.pop(1)       # removes 'ORD-103'
print(cancelled, orders)        # ORD-103 ['ORD-101', 'ORD-102']

# DELETE entire list contents
orders.clear()                  # []
```

Note:
- Wrong: `remove(x)` when `x` is not in the list → `ValueError`.
- Prefer: `if x in lst: lst.remove(x)`.

Remove the first ORD-2 from an orders list with duplicate entries.

```
orders = ["ORD-1", "ORD-2", "ORD-2"]
orders.remove("ORD-2")
print(orders)
```

Expected Output:

```
['ORD-1', 'ORD-2']
```

Pop the last task and print what was removed and the remaining tasks.

```
tasks = ["Login page", "API", "Deploy"]
cancelled = tasks.pop()
print(cancelled)
print(tasks)
```

Expected Output:

```
Deploy
['Login page', 'API']
```

Step 4 — Useful list methods

```
marks = [78, 82, 91, 65, 88, 82]

print(marks.count(82))          # 2 — how many times 82 appears
print(marks.index(91))          # 2 — index of first 91
```

```
marks.sort()                    # ascending: [65, 78, 82, 82, 88, 91]
marks.reverse()                 # reverses current order in place

# sorted() returns NEW list; original unchanged
scores = [88, 72, 95]
top_sorted = sorted(scores, reverse=True)
print(top_sorted)               # [95, 88, 72]
print(scores)                   # [88, 72, 95] — unchanged
```

Note:
- Wrong: Confusing `sort()` with `sorted()` — unexpected in-place change.
- Prefer: `sort()` changes the original; `sorted()` returns a new list.

Print how many times 82 appears in marks and the index of 91.

```
marks = [78, 82, 91, 65, 88, 82]
print(marks.count(82))
print(marks.index(91))
```

Create a descending sorted copy of scores without changing the original.

```
original = [88, 72, 95]
ranked = sorted(original, reverse=True)
print(ranked)
print(original)  # unchanged
```

Step 5 — Loop through a list

```
student_names = ["Aisha", "Dev", "Meera"]
student_marks = [78, 92, 85]

# Loop with index using range
for i in range(len(student_names)):
    print(f"{student_names[i]} scored {student_marks[i]}")

# Pythonic: zip two parallel lists
for name, mark in zip(student_names, student_marks):
    print(f"{name}: {mark}%")
```

Note:
- Wrong: Modifying a list while looping over it — skipped items or infinite loops.
- Prefer: Loop over a copy: `for x in lst[:]`.

Zip names and marks pairs and print each as name: mark%.

```
for name, mark in zip(["Priya", "Ravi"], [88, 76]):
    print(f"{name}: {mark}%")
```

Loop expenses and print only amounts greater than ₹500.

```
for amount in [200, 1500, 75]:
    if amount > 500:
        print(f"₹{amount}")
```

Step 6 — Nested lists (table-like data)

```
# Each inner list = one student: [name, maths, english, science]
students = [
    ["Aisha", 78, 82, 91],
    ["Dev",   92, 88, 85],
    ["Meera", 65, 70, 88],
]

# READ nested value — Meera's English mark (index 2)
print(students[2][2])           # 70

# UPDATE — fix Dev's maths mark
students[1][1] = 94
print(students[1])              # ['Dev', 94, 88, 85]

# E-commerce: order with line items [product, qty, price]
order_lines = [
    ["Notebook", 2, 120],
    ["Pen", 5, 15],
]
total = sum(line[1] * line[2] for line in order_lines)
print(f"Order total: ₹{total}")   # ₹315
```

Look up Ravi's second subject mark from a nested list of rows.

```
rows = [["Priya", 78, 82], ["Ravi", 91, 70]]
print(rows[1][2])  # Ravi's second subject = 70
```

Compute per-line and total order cost from nested [name, qty, price] rows.

```
order = [["Chai", 2, 20], ["Samosa", 3, 25]]
total = 0
for name, qty, price in order:
    line = qty * price
    total += line
    print(f"{name}: ₹{line}")
print(f"Total: ₹{total}")
```

Expected Output:

```
Chai: ₹40
Samosa: ₹75
Total: ₹115
```

Step 7 — Introduction to list comprehensions

```
expenses = [450, 1200, 89, 2500, 320, 890]

# Build new list: only expenses above ₹500
high_expenses = [amt for amt in expenses if amt > 500]
print(high_expenses)            # [1200, 2500, 890]

# Transform: apply 18% GST to product prices
prices = [100, 250, 499]
```

```
prices_with_gst = [round(p * 1.18, 2) for p in prices]
print(prices_with_gst)          # [118.0, 295.0, 588.82]

# Employee IDs as strings for display
emp_ids = [101, 102, 105]
emp_labels = [f"EMP-{eid}" for eid in emp_ids]
print(emp_labels)               # ['EMP-101', 'EMP-102', 'EMP-105']
```

Build a list comprehension of expenses above ₹500.

```
expenses = [450, 1200, 89, 2500, 320]
big = [a for a in expenses if a > 500]
print(big)
```

Apply 18% GST to each price with a list comprehension.

```
prices = [100, 250, 499]
with_gst = [round(p * 1.18, 2) for p in prices]
print(with_gst)
```

Step 8 — List copying (avoid surprise bugs)

```
original = [1, 2, 3]
alias = original          # both names point to SAME list
alias.append(4)
print(original)           # [1, 2, 3, 4] — surprise if unintended!

# Safe shallow copy
copy1 = original.copy()
copy2 = list(original)
copy3 = original[:]
```

Note:
- Wrong: Relying on a shallow copy of a *nested* list (inner lists still shared); using `is` to compare list *content* (`[1, 2] is [1, 2]` is False).
- Prefer: `copy.deepcopy()` for deep copy (advanced); use `==` for value equality, `is` only for the same object.

Demonstrate the alias bug: append via b changes a.

```
a = [1, 2]
b = a
b.append(3)
print(a)  # [1, 2, 3] — same list
```

Expected Output:

```
[1, 2, 3]
```

Remember: `b = a` does **not** copy the list — both names point to the same object. Use `.copy()` / `list(a)` / `a[:]` when you need a separate list.

Use .copy() so appending to b does not change a.

```
a = [1, 2]
b = a.copy()
b.append(3)
print(a)  # [1, 2]
print(b)  # [1, 2, 3]
```

Practice Exercises

Basic

1. Create a list of 5 product names. Print first, last, and middle item.

Model answer:

```
products = ["Notebook", "Pen", "USB Drive", "Mouse", "Keyboard"]
print(products[0])   # first
print(products[-1])  # last
print(products[2])   # middle (index 2 of 5)
```

2. Add two more products using `append` and one using `insert(0, ...)`.

Model answer:

```
products = ["Notebook", "Pen", "USB Drive", "Mouse", "Keyboard"]
products.append("Charger")
products.append("Headset")
products.insert(0, "Laptop Bag")
print(products)
```

3. Given `marks = [55, 78, 92, 67, 88]`, print the highest mark using `max()` and count how many scored above 80.

Model answer:

```
marks = [55, 78, 92, 67, 88]
print("Highest:", max(marks))
count = 0
for m in marks:
    if m > 80:
        count += 1
print("Above 80:", count)  # 2 → 92 and 88
```

Intermediate

4. Store 3 employees as `[id, name, department]`. Update one department and print all as formatted lines.

Model answer:

```
employees = [
    [101, "Ravi", "HR"],
    [102, "Priya", "IT"],
    [103, "Arjun", "Sales"],
]
employees[1][2] = "Finance"  # Priya moves to Finance
```

```
for emp in employees:
    print(f"ID {emp[0]} | {emp[1]} | {emp[2]}")
```

5. From `expenses = [200, 1500, 75, 890, 1200, 45]`, create a new list of expenses above ₹300 using a list comprehension.

Model answer:

```
expenses = [200, 1500, 75, 890, 1200, 45]
big = [e for e in expenses if e > 300]
print(big)  # [1500, 890, 1200]
```

6. Remove all occurrences of `"Pen"` from `cart = ["Book", "Pen", "Pen", "Eraser"]` using a loop (not `remove` in a naive loop).

Model answer: Build a new list (safe — mutating while iterating + repeated `remove` is error-prone):

`python cart = ["Book", "Pen", "Pen", "Eraser"] cleaned = [] for item in cart: if item != "Pen": cleaned.append(item) cart = cleaned print(cart) # ['Book', 'Eraser']`

Interview-style

7. Reverse a list **without** using `reverse()` or `[::-1]` — use a loop.

Model answer:

```
nums = [10, 20, 30, 40]
reversed_list = []
for i in range(len(nums) - 1, -1, -1):
    reversed_list.append(nums[i])
print(reversed_list)  # [40, 30, 20, 10]
```

8. Find the second-largest mark in a list (handle duplicates).

Model answer:

```
marks = [88, 92, 92, 67, 88]
unique_sorted = sorted(set(marks), reverse=True)
print(unique_sorted[1])  # 88
```

Or without relying on `set` only for uniqueness walk:

`python marks = [88, 92, 92, 67, 88] first = second = float("-inf") for m in marks: if m > first: second = first first = m elif first > m > second: second = m print(second) # 88`

9. Flatten one level: `[[1, 2], [3], [4, 5]]` → `[1, 2, 3, 4, 5]`.

Model answer:

```
nested = [[1, 2], [3], [4, 5]]
flat = []
for row in nested:
    for x in row:
        flat.append(x)
print(flat)
```

Mini-Project (Assignment): Student Marks Manager (CLI)

Title: Menu-driven student marks CRUD

Lab kit (build here): 15_Lab_Assets/10_marks_manager/

What to build

```
=== Student Marks Manager ===
1. Add student mark
2. View all marks
3. Show class average
4. List students above 80
5. Exit
```

Data model: parallel lists `names` and `marks`, or nested list `[name, mark]`.

Starter structure:

```
students = []   # each item: [name, mark]

while True:
    print("\n1.Add  2.View  3.Average  4.Above 80  5.Exit")
    choice = input("Choice: ")
    if choice == "1":
        name = input("Name: ")
        mark = int(input("Mark: "))
        students.append([name, mark])
    elif choice == "2":
        for s in students:
            print(f"{s[0]}: {s[1]}")
    # ... complete remaining options
```

Stretch

- Show topper name(s) for the highest mark.
- Reject marks outside 0–100.

Done when

- [ ] Lab kit marks-manager ticket CRUD + toppers works
- [ ] Menu options 1–4 behave correctly; 5 exits
- [ ] Average and “above 80” use the current list data

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why does Python use index `0` for the first element? How is that different from "row 1" in Excel?
2. When would you use a list of strings vs a list of dictionaries for **Employee Management Portal** employee data? (Preview of dict module.)
3. Is `[1, 2, 3] + [4, 5]` the same as extending? What does each return?
4. Why might modifying a list while iterating over it cause bugs?
5. In a **Swiggy** order, what part would be a list vs a single value?

6. **Is Python list thread-safe?**
   A: Not for concurrent writes; for entry-level interviews: lists are fine in single-threaded scripts; multi-user apps usually store shared data in a database.
7. **List vs array (NumPy)?**
   A: Lists hold mixed types; NumPy arrays are homogeneous and faster for numeric work.
8. **Time complexity of `x in my_list`?**
   A: O(n) linear search; sets are O(1) average for membership.
9. **Shallow vs deep copy?**
   A: Shallow copies outer list; nested inner lists may still be shared.
10. **How do you remove duplicates preserving order?**
    A: Loop and append to new list if not seen; or use `dict.fromkeys()` trick.
